

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
КАФЕДРА ІНФОРМАТИКИ ТА ПРОГРАМНОЇ ІНЖЕНЕРІЇ

КУРСОВА РОБОТА

з дисципліни «Аналіз даних в інформаційних системах»

на тему: «Аналіз та прогнозування ймовірності затримки доставки замовлення на основі
логістичних параметрів та географічного розташування»

Студента 2 курсу ІП-45 групи
Спеціальності: 121
«Інженерія програмного забезпечення»
Янова Богдана

«ПРИЙНЯВ» з оцінкою

доц. Ліхоузова Т.А. / доц. Олійник Ю.О.

Підпис

Дата

Київ - 2026 рік

Національний технічний університет України “КПІ ім. Ігоря Сікорського”
Кафедра інформатики та програмної інженерії
Дисципліна Аналіз даних в інформаційно-управляючих системах
Спеціальність 121 "Інженерія програмного забезпечення"

Курс 2 Група ІІ-45

Семестр 4

ЗАВДАННЯ
на курсову роботу студента
Янова Богдана

1.Тема роботи Аналіз та прогнозування ймовірності затримки доставки замовлення на основі логістичних параметрів та географічного розташування

2.Строк здачі студентом закінченої роботи 19.06.2026

3. Вхідні дані до роботи методичні вказівки до курсової роботи, обрані дані з сайту <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

4.Зміст розрахунково-пояснювальної записки (перелік питань, які підлягають розробці)

1.Постановка задачі

2.Аналіз предметної області

3.Розробка сховища даних

4.Інтелектуальний аналіз даних

5.Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

6.Дата видачі завдання 16.04.2026

КАЛЕНДАРНИЙ ПЛАН

№ п/п	Назва етапів курсової роботи	Термін виконання етапів роботи	Підписи керівника, студента
1.	Отримання теми курсової роботи	16.04.2026	
2.	Визначення зовнішніх джерел даних	16.04.2026	
3.	Пошук та вивчення літератури з питань курсової роботи	16.04.2026	
4.	Розробка моделі сховища даних	16.04.2026	
4.	Розробка ETL процесів	16.04.2026	
5.	Обґрунтування методів інтелектуального аналізу даних	16.04.2026	
6.	Застосування та порівняння ефективності методів інтелектуального аналізу даних	16.04.2026	
7.	Підготовка пояснювальної записки	16.04.2026	
8.	Здача курсової роботи на перевірку	16.04.2026	
9.	Захист курсової роботи	16.05.2026	

Студент

(підпис)

Янов Б.Э.

(прізвище, ім'я, по батькові)

Керівник

(підпис)

доц. Ліхоузова Т.А.

(прізвище, ім'я, по батькові)

Керівник

(підпис)

доц. Олійник Ю.О.

(прізвище, ім'я, по батькові)

АНОТАЦІЯ

Пояснювальна записка до курсової роботи присвячена аналізу та прогнозуванню ймовірності затримки доставки замовлення в електронній комерції. У роботі використано публічний набір даних Olist, який містить інформацію про замовлення, покупців, продавців, товари, платежі, географічні координати та фактичні строки доставки.

Об'єкт дослідження: процес доставки замовлень в електронній комерції.

Предмет дослідження: моделі класифікації для прогнозування факту затримки доставки на основі логістичних, платіжних, товарних і географічних параметрів.

Мета роботи: побудувати та порівняти моделі, які оцінюють ймовірність того, що замовлення буде доставлене пізніше очікуваної дати.

У роботі використано результати попередньої лабораторної роботи: stage-зону, виміри та фактові таблиці SQLite-сховища Olist. Для моделювання сформовано аналітичну вибірку на рівні замовлення, побудовано ознаки відстані між продавцем і покупцем, строку очікуваної доставки, параметрів оплати, характеристик товарів та регіональних маршрутів. Розглянуто чотири методи класифікації: логістичну регресію, дерево рішень, випадковий ліс і градієнтний бустинг. Для оцінювання використано часовий поділ на навчальну, валідаційну та тестову вибірки.

За результатами експериментів найкращою за average precision та ROC-AUC стала логістична регресія. Отримана модель має обмежену, але практично інтерпретовану прогнозну здатність: вона краще виявляє ризикові замовлення, ніж випадковий підхід, однак потребує додаткових операційних даних перевізників для високоточного продукційного використання.

Пояснювальна записка містить 4 розділи, 7 таблиць, 7 рисунків і 10 джерел.

СХОВИЩЕ ДАНИХ, ETL, OLIST, ЕЛЕКТРОННА КОМЕРЦІЯ, ЛОГІСТИКА, ЗАТРИМКА ДОСТАВКИ, КЛАСИФІКАЦІЯ, ЛОГІСТИЧНА РЕГРЕСІЯ, ВИПАДКОВИЙ ЛІС, ГРАДІЄНТНИЙ БУСТИНГ.

ЗМІСТАНОТАЦІЯ 4

ВСТУП	6
1. ПОСТАНОВКА ЗАДАЧІ	7
1.1 Мета роботи.....	7
1.2 Об'єкт і предмет дослідження	7
1.3 Формалізація задачі.....	7
1.4 Задачі дослідження.....	7
2. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ	9
2.1 Огляд предметної області.....	9
2.2 Джерело даних	9
2.3 Особливості даних.....	10
2.4 Постановка аналітичних питань	10
3. ВИКОРИСТАННЯ СХОВИЩА ДАНИХ ТА ПІДГОТОВКА ВИБІРКИ	12
3.1 Використане сховище даних	12
3.2 ETL-процес	13
3.3 Формування аналітичної вибірки	13
3.4 Побудовані ознаки	14
3.5 Розділення даних	16
4. ІНТЕЛЕКТУАЛЬНИЙ АНАЛІЗ ДАНИХ	18
4.1 Обґрунтування вибору методів	18
4.2 Метрики якості.....	18
4.3 Практична реалізація.....	19
4.4 Порівняння ефективності моделей.....	20
4.5 Аналіз найкращої моделі	21
4.6 Інтерпретація результатів.....	22
ВИСНОВКИ.....	24
ПЕРЕЛІК ПОСИЛАНЬ	25
ДОДАТОК А. ТЕКСТИ ПРОГРАМНОГО КОДУ	26

ВСТУП

Своєчасна доставка є одним із ключових чинників якості сервісу в електронній комерції. Для покупця затримка означає гірший досвід взаємодії з магазином, а для маркетплейсу або продавця - додаткові звернення до служби підтримки, компенсації, погіршення рейтингу та ризик втрати повторних продажів. Тому завдання раннього виявлення замовлень з підвищеною ймовірністю затримки має практичну цінність для операційної аналітики.

У межах цієї курсової роботи розглянуто задачу бінарної класифікації: для кожного замовлення необхідно спрогнозувати, чи буде воно доставлене пізніше очікуваної дати. На відміну від задачі аналізу вже завершених доставок, прогнозування повинно спиратися лише на ознаки, доступні на момент оформлення або ранньої обробки замовлення: географію покупця й продавця, відстань між ними, характеристики товарів, оплату, дату купівлі та очікуваний строк доставки.

Робота спирається на вже виконаний ETL із попередньої лабораторної роботи. Сирі CSV-файли Olist [1] були завантажені до stage-зони, після чого трансформовані у виміри та фактові таблиці SQLite-сховища. У цій курсовій не створювався окремий незалежний ETL «з нуля»: наявне сховище використано як основне джерело даних, а додатковий Python-скрипт формує тільки аналітичну вибірку для моделей.

Теоретичну основу роботи становлять праці з проєктування сховищ даних, методів класифікації [2; 3; 4] та оцінювання моделей на незбалансованих даних [5; 6].

1. ПОСТАНОВКА ЗАДАЧІ

1.1 Мета роботи

Метою курсової роботи є побудова та порівняння моделей прогнозування ймовірності затримки доставки замовлення на основі логістичних параметрів, географічного розташування покупця і продавця та характеристик замовлення.

1.2 Об'єкт і предмет дослідження

Об'єктом дослідження є процес виконання та доставки замовлень в електронній комерції.

Предметом дослідження є методи інтелектуального аналізу даних, які дають змогу класифікувати замовлення за ризиком затримки доставки.

1.3 Формалізація задачі

Нехай для кожного замовлення задано вектор ознак

$$X = (x_1, x_2, \dots, x_n), \quad (1.1)$$

де ознаки описують товар, продавця, покупця, маршрут, оплату та часові параметри замовлення. Необхідно побудувати функцію

$$f(X) = P(y = 1 | X), \quad (1.2)$$

де $y = 1$ означає, що замовлення доставлено із затримкою, а $y = 0$ - що замовлення доставлено вчасно або раніше очікуваної дати.

Цільова ознака визначається на основі порівняння фактичної дати доставки з очікуваною датою:

$$y = \{ 1, \text{ якщо } \text{delivery_date} > \text{estimated_delivery_date}; 0, \text{ якщо } \text{delivery_date} \leq \text{estimated_delivery_date}. \} \quad (1.3)$$

Важливо, що фактична дата доставки, кількість днів доставки та фактична кількість днів затримки не використовуються як вхідні ознаки моделі, оскільки вони стають відомими тільки після завершення доставки. Ці поля використовуються лише для формування цільової змінної та описової статистики.

1.4 Задачі дослідження

Для досягнення мети необхідно виконати такі задачі:

1. Проаналізувати предметну область доставки замовлень в електронній комерції.
2. Описати джерело даних та використане сховище Olist.
3. Сформуванати аналітичну вибірку на рівні замовлення.
4. Побудувати логістичні, товарні, платіжні та географічні ознаки.
5. Обрати та обґрунтувати щонайменше три методи класифікації.

6. Розділити дані на навчальну, валідаційну та тестову вибірки з урахуванням часу.
7. Навчити моделі, підібрати пороги класифікації та оцінити якість.
8. Порівняти результати й зробити висновок щодо практичної придатності моделей.

2. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

2.1 Огляд предметної області

Електронна комерція об'єднує продавців, покупців, платіжні сервіси, склади, логістичних операторів і служби підтримки. Після оформлення замовлення система має виконати кілька етапів: підтвердити оплату, передати замовлення продавцю, підготувати товар, відправити його перевізнику та доставити покупцю. На кожному етапі можливі затримки.

На ймовірність затримки можуть впливати такі фактори:

- географічна віддаленість продавця і покупця;
- регіональні особливості логістичної інфраструктури;
- очікуваний строк доставки, який встановлюється під час оформлення замовлення;
- тип і вартість товару;
- вага та габарити товару;
- кількість позицій у замовленні;
- тип оплати та кількість платежів;
- сезонність і день тижня оформлення замовлення.

Для бізнесу прогноз ризику затримки може використовуватися в таких сценаріях:

- пріоритезація проблемних замовлень у логістичному моніторингу;
- раннє інформування клієнтів про можливі зміни строків;
- контроль якості роботи продавців і перевізників;
- коригування правил розрахунку очікуваних дат доставки;
- побудова внутрішніх KPI для операційних команд.

2.2 Джерело даних

У роботі використано Brazilian E-Commerce Public Dataset by Olist [1]. Це публічний набір даних з інформацією про приблизно 100 тисяч замовлень бразильського маркетплейсу Olist за 2016-2018 роки. Дані містять окремі CSV-файли для замовлень, товарних позицій, покупців, продавців, платежів, відгуків, товарів, категорій та геолокації.

Основні таблиці джерела наведено в таблиці 2.1.

Таблиця 2.1 - Основні таблиці джерела даних Olist

Файл	Зміст
olist_orders_dataset.csv	статуси замовлень, дата купівлі, дата підтвердження, фактична та очікувана дата

	доставки
olist_order_items_dataset.csv	товарні позиції, продавці, ціна, фрахт, гранична дата відправлення
olist_customers_dataset.csv	покупці, міста, штати, ZIP-prefix
olist_sellers_dataset.csv	продавці, міста, штати, ZIP-prefix
olist_products_dataset.csv	категорія, вага, габарити, довжина опису, кількість фото
olist_geolocation_dataset.csv	координати ZIP-prefix для побудови географічних ознак
olist_order_payments_dataset.csv	типи оплати, кількість платежів, сума оплати
product_category_name_translation.csv	переклад назв категорій англійською

2.3 Особливості даних

Набір Olist є корисним для аналізу логістики, оскільки в ньому одночасно присутні:

- фактичні та очікувані дати доставки;
- географія покупця і продавця;
- характеристики товару;
- параметри платежу;
- зв'язок між замовленням, продавцем і товарними позиціями.

Разом з тим дані мають обмеження. У них немає прямої інформації про конкретного перевізника, склад, погодні умови, завантаженість логістичних центрів, фактичний маршрут або операційні інциденти. Тому очікується, що модель може виявляти загальні закономірності, але не зможе пояснити всі випадки затримок.

2.4 Постановка аналітичних питань

Проведений аналіз має відповісти на такі питання:

9. Яка частка доставлених замовлень має затримку?
10. Чи відрізняється ризик затримки за географічними напрямками?
11. Чи пов'язана відстань між продавцем і покупцем із часткою затримок?
12. Які ознаки мають найбільший вплив на прогноз моделі?
13. Який із розглянутих методів класифікації найкраще підходить для задачі?

14. Чи можна використовувати побудовану модель як інструмент раннього попередження?

3. ВИКОРИСТАННЯ СХОВИЩА ДАНИХ ТА ПІДГОТОВКА ВИБІРКИ

3.1 Використане сховище даних

У попередній лабораторній роботі було створено SQLite-сховище Olist у директорії `lab_olist_warehouse`. Воно містить stage-таблиці, виміри, фактові таблиці, SQL-скрипти трансформацій і звіт з ETL. У цій курсовій роботі це сховище використовується як основне джерело аналітичних даних.

Проектування сховища виконано відповідно до принципів багатовимірного моделювання Кімбола та концепції корпоративного сховища Інмона.

Основні файли попередньої роботи наведено в таблиці 3.1.

Таблиця 3.1 - Файли попередньої лабораторної роботи

Файл	Призначення
<code>lab_olist_warehouse/scripts/etl_olist.py</code>	Python ETL для завантаження CSV у stage-зону та сховище
<code>lab_olist_warehouse/sql/01_stage_schema.sql</code>	опис stage-таблиць
<code>lab_olist_warehouse/sql/02_warehouse_schema.sql</code>	опис вимірів і фактових таблиць
<code>lab_olist_warehouse/sql/03_transform_load.sql</code>	SQL-перетворення та завантаження у сховище
<code>lab_olist_warehouse/sql/04_analysis_queries.sql</code>	прикладі аналітичних запитів
<code>lab_olist_warehouse/output/olist_dw.sqlite</code>	готове SQLite-сховище

Сховище має зіркоподібну структуру (точніше - сузір'я фактів зі спільними вимірами) з фактовими таблицями `fact_order_items`, `fact_payments`, `fact_reviews` і вимірами `dim_customer`, `dim_seller`, `dim_product`, `dim_category`, `dim_location`, `dim_date`, `dim_order_status`, `dim_payment_type`. Структуру наведено на рисунку 3.1.

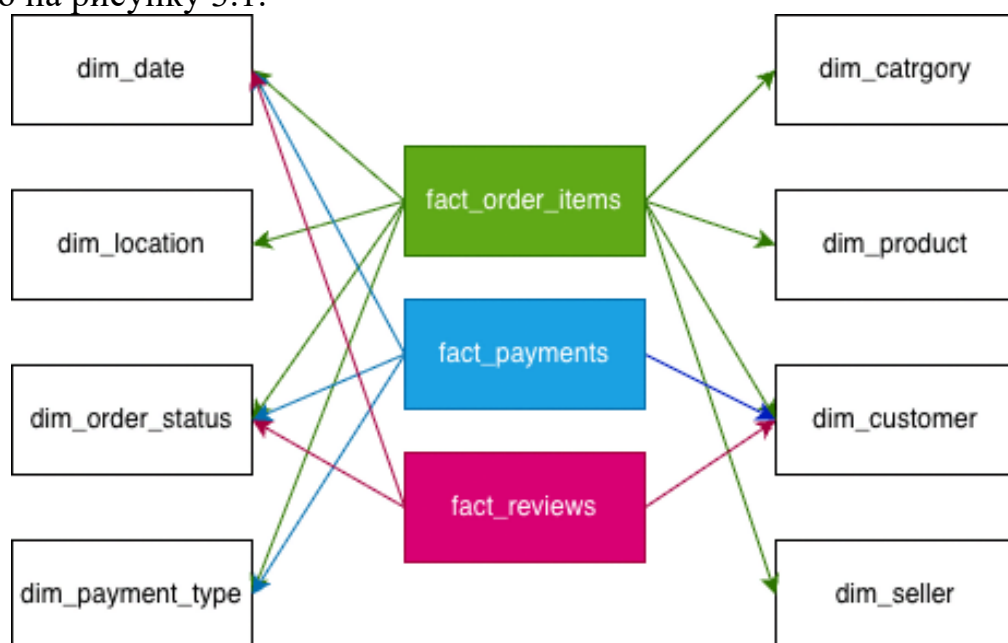


Рисунок 3.1 - Модель використаного сховища даних

3.2 ETL-процес

ETL-процес (Extract - Transform - Load) складається з таких кроків:

15. Завантаження сирих CSV-файлів до stage-таблиць (Extract).
16. Нормалізація назв міст, штатів, категорій та статусів (Transform).
17. Побудова виміру дат на основі всіх дат замовлень, доставок, платежів і відгуків.
18. Агрегація геолокацій за ZIP-prefix (одному префіксу зіставляються середні координати).
19. Завантаження вимірів покупців, продавців, товарів, категорій і типів оплати (Load).
20. Формування фактової таблиці товарних позицій із розрахунком `days_to_delivery`, `delivery_delay_days`, `was_delivered_late`.
21. Формування фактових таблиць платежів і відгуків.

Повторне завантаження реалізовано ідемпотентно через природні бізнес-ключі (`order_id`, `product_id`, `customer_id` тощо), тому повторний запуск ETL не створює дублікатів.

Для курсової роботи ETL не дублювався. Було використано результат попередньої лабораторної роботи, а новий скрипт `course_work/scripts/analyze_delivery_delay.py` виконує тільки побудову аналітичної вітрини на рівні замовлення та навчання моделей.

3.3 Формування аналітичної вибірки

Одиницею спостереження у моделюванні є замовлення. Оскільки в Olist одне замовлення може містити кілька товарних позицій, дані з `fact_order_items` агреговано за `order_id`. Додатково приєднано агреговану інформацію про платежі.

До вибірки включено лише замовлення, для яких відома цільова ознака `was_delivered_late`. Узагальнену статистику отриманої вибірки наведено в таблиці 3.2.

Таблиця 3.2 - Узагальнена статистика аналітичної вибірки

Показник	Значення
Кількість замовлень	96 476
Кількість полів після підготовки	49
Частка замовлень із затримкою	8.11 %

Середня затримка серед запізнілих доставок	9.55 дня
Медіанна затримка серед запізнілих доставок	5.81 дня
Медіанна відстань між продавцем і покупцем	434.86 км
Медіанна відстань для запізнілих доставок	512.45 км
Медіанна відстань для доставок без затримки	427.22 км

Розподіл цільової ознаки наведено на рисунку 3.2.



Рисунок 3.2 - Розподіл замовлень за фактом затримки доставки
Цільова ознака є незбалансованою: замовлень із затримкою суттєво менше, ніж доставок без затримки. Це впливає на вибір метрик оцінювання. Звичайна ассигасу може бути оманливою, оскільки модель, яка майже завжди прогнозує «без затримки», отримає високу точність, але не буде корисною для виявлення ризикових замовлень [5].

3.4 Побудовані ознаки

У моделі використано групи ознак, наведені в таблиці 3.3.

Таблиця 3.3 - Групи ознак моделі

Група	Приклади ознак
Географічні	штат покупця, штат продавця, пара штатів маршруту, відстань між координатами продавця і покупця, ознака одного штату

Логістичні	очікувана кількість днів до доставки, кількість днів до shipping limit, день тижня очікуваної доставки
Товарні	кількість позицій, середня вага, середній об'єм, габарити, кількість фото, категорія товару
Вартісні	сума товарів, сума фрахту, загальна сума замовлення, співвідношення фрахту до ціни
Платіжні	тип оплати, кількість платежів, загальна сума оплати
Часові	місяць купівлі, тиждень року, день тижня купівлі

Відстань між покупцем і продавцем розраховано за формулою гаверсинуса на основі широти та довготи ZIP-prefix:

$$d = 2R \sqrt{\frac{(\Delta \varphi)^2}{4} + \frac{\varphi_1 \varphi_2 (\Delta \lambda)^2}{4}}, \quad (3.1)$$

де R - радіус Землі (6371 км), φ - широта, λ - довгота, $\Delta \varphi$ та $\Delta \lambda$ - різниці координат точок.

Не використовувалися як вхідні ознаки:

- days_to_delivery;
- delivery_delay_days;
- фактична дата доставки клієнту;
- дата передачі перевізнику.

Ці поля не можна використовувати для прогнозу на момент оформлення замовлення, бо вони створюють витік цільової інформації (data leakage).

Штати покупців з найбільшою часткою затримок наведено на рисунку 3.3.

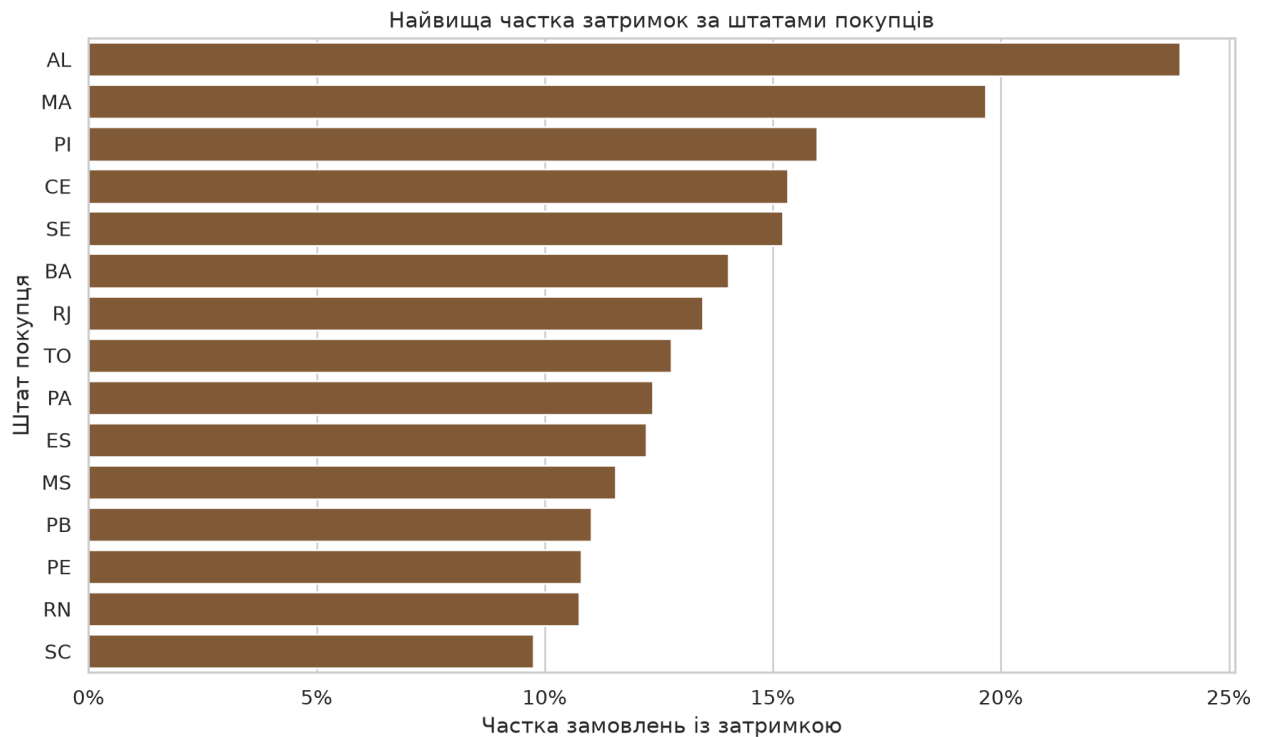


Рисунок 3.3 - Штати покупців з найбільшою часткою затримок
Залежність частки затримок від відстані наведено на рисунку 3.4.

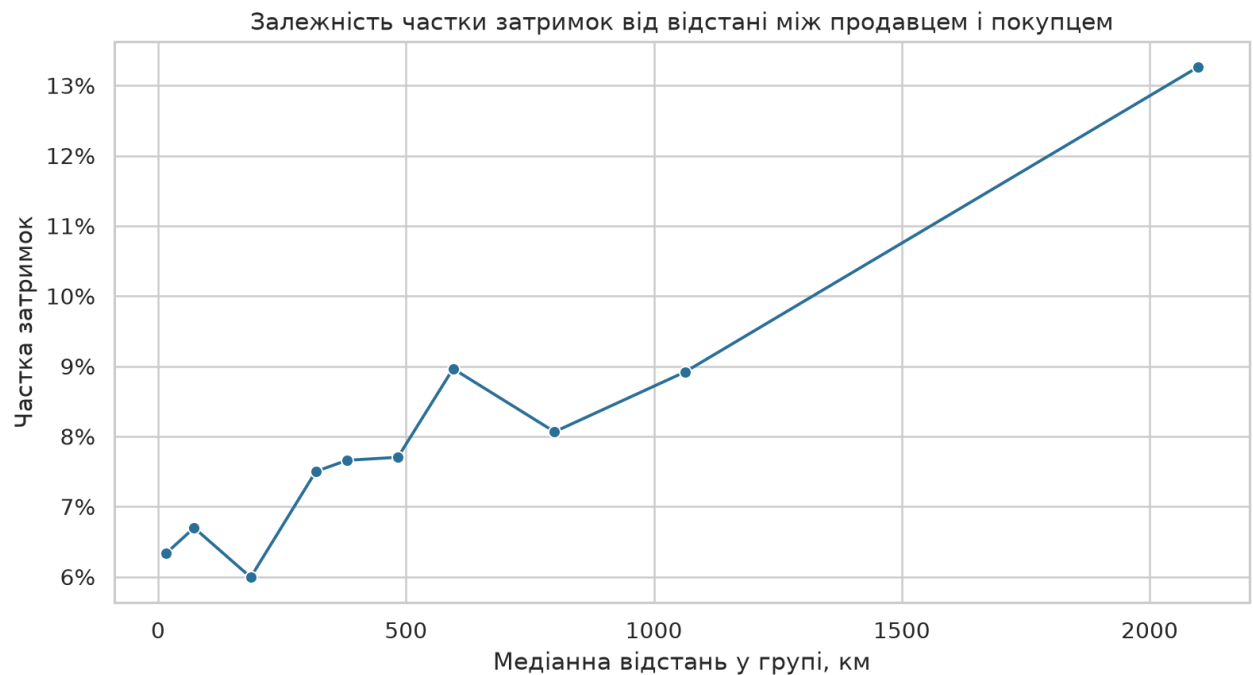


Рисунок 3.4 - Залежність частки затримок від відстані між продавцем і покупцем

Графік на рисунку 3.4 показує, що відстань сама по собі не пояснює всі затримки, але для запізнених доставок медіанна відстань вища. Це підтверджує доцільність включення географічних ознак до моделі.

3.5 Розділення даних

Оскільки задача має прогнозний характер, дані розділено не випадково, а за часом купівлі. Такий хронологічний поділ імітує реальний сценарій

застосування: модель навчається на минулому й оцінюється на майбутньому періоді [6]. Параметри поділу наведено в таблиці 3.4.

Таблиця 3.4 - Розділення даних на вибірки

Вибірка	Кількість замовлень	Період
Навчальна	67 533	15.09.2016 - 15.04.2018
Валідаційна	9 647	15.04.2018 - 26.05.2018
Тестова	19 296	26.05.2018 - 29.08.2018

Навчальна вибірка використовується для оцінювання параметрів моделей. Валідаційна вибірка використовується для підбору порогу класифікації за F1-мірою. Тестова вибірка використовується тільки для фінального порівняння якості.

4. ІНТЕЛЕКТУАЛЬНИЙ АНАЛІЗ ДАНИХ

4.1 Обґрунтування вибору методів

Задача прогнозування затримки доставки є задачею бінарної класифікації з незбалансованими класами. Для неї обрано чотири методи, які мають різну природу та дають змогу порівняти лінійний і нелінійні підходи [3].

Логістична регресія використовується як інтерпретована базова модель. Вона оцінює ймовірність належності до класу затримки через логістичну функцію [2]:

$$P(y=1 | X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)}}. \quad (4.1)$$

Перевага методу полягає у простоті, швидкості навчання та можливості інтерпретувати коефіцієнти ознак. Недолік - обмежена здатність описувати складні нелінійні взаємодії.

Дерево рішень будує послідовність правил поділу простору ознак. Метод добре інтерпретується та може моделювати нелінійні залежності, але схильний до перенавчання, тому в роботі обмежено максимальну глибину дерева (`max_depth = 12`) та мінімальну кількість об'єктів у листі (`min_samples_leaf = 50`).

Випадковий ліс є ансамблем дерев рішень [3]. Кожне дерево навчається на випадковій підвибірці даних та ознак (бутстреп-агрегування, `bagging`), а результат формується агрегуванням прогнозів. Метод зазвичай стійкіший за окреме дерево, але менш інтерпретований і може гірше працювати при часовому зсуві розподілу.

Гرادієнтний бустинг послідовно будує ансамбль слабких моделей, де кожна наступна модель намагається виправити помилки попередніх (`boosting`) [4]. Метод часто добре працює на табличних даних, але потребує уважного налаштування параметрів і може бути чутливим до зміни розподілу між навчальною та тестовою вибірками.

Принципова відмінність двох ансамблевих методів: випадковий ліс будує дерева **незалежно** й усереднює їх, зменшуючи дисперсію; градієнтний бустинг будує дерева **послідовно**, зменшуючи зміщення.

4.2 Метрики якості

Через незбалансованість класів використано кілька метрик [6]:

- `accuracy` - частка правильних прогнозів;
- `balanced_accuracy` - середнє значення `recall` для обох класів;
- `precision` - частка справді запізнілих серед замовлень, позначених моделлю як ризикові;
- `recall` - частка знайдених запізнілих замовлень серед усіх фактично запізнілих;
- `F1` - гармонічне середнє `precision` і `recall`;

- ROC-AUC - якість ранжування класів за всіма можливими порогами;
- `average_precision` - площа під precision-recall кривою, корисна для незбалансованих задач [5].

Метрики precision, recall і F1 обчислюються за матрицею помилок:

$$\text{Precision} = (\text{TP}) / (\text{TP} + \text{FP}), \text{ Recall} = (\text{TP}) / (\text{TP} + \text{FN}), \\ \text{F}_1 = 2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall}), \quad (4.2)$$

де *TP* - істинно позитивні, *FP* - хибно позитивні, *FN* - хибно негативні прогнози. Основними метриками для порівняння вважаються `average_precision`, ROC-AUC, `balanced_accuracy` та F1. Ассурасу наводиться довідково, але не є головним критерієм.

4.3 Практична реалізація

Практичну частину реалізовано у файлі

`course_work/scripts/analyze_delivery_delay.py` з використанням бібліотеки `scikit-learn` [6; 7]. Скрипт виконує такі дії:

22. Читає дані зі SQLite-сховища `lab_olist_warehouse/output/olist_dw.sqlite`.
23. Агрегує товарні позиції та платежі до рівня замовлення.
24. Будує додаткові ознаки: відстань, маршрут штатів, співвідношення фрахту до ціни, об'єм товару, строки очікуваної доставки.
25. Вилучає ознаки, які створюють витік даних.
26. Ділить дані на `train`, `validation` і `test` за часом купівлі.
27. Кодує категоріальні ознаки за допомогою `one-hot encoding`.
28. Заповнює пропуски медіаною (числові ознаки) або найчастішим значенням (категоріальні).
29. Навчає чотири моделі класифікації.
30. Підбирає поріг класифікації на валідаційній вибірці.
31. Оцінює якість на тестовій вибірці.
32. Зберігає метрики, графіки та підготовлений датасет у `course_work/assets`.

Усі етапи попередньої обробки (масштабування, заповнення пропусків, кодування) інкапсульовано в конвеєр (`Pipeline`) і навчено виключно на навчальній вибірці, що додатково унеможлиблює витік даних. Для врахування

дисбалансу класів у моделях використано `class_weight='balanced'` або аналогічні ваги класів.

4.4 Порівняння ефективності моделей

Фінальні результати на тестовій вибірці наведено у таблиці 4.1.

Таблиця 4.1 - Порівняння моделей прогнозування затримки доставки

Модель	Поріг	Accuracy	Balance d accuracy	Precision	Recall	F1	ROC-AUC	Average precision
Логістична регресія	0.749	0.641	0.655	0.094	0.672	0.165	0.705	0.104
Гرادієнтний бустинг	0.485	0.790	0.534	0.071	0.247	0.111	0.652	0.078
Випадковий ліс	0.454	0.812	0.511	0.060	0.174	0.089	0.520	0.059
Дерево рішень	0.645	0.814	0.512	0.061	0.175	0.090	0.457	0.056

Візуальне порівняння метрик наведено на рисунку 4.1.

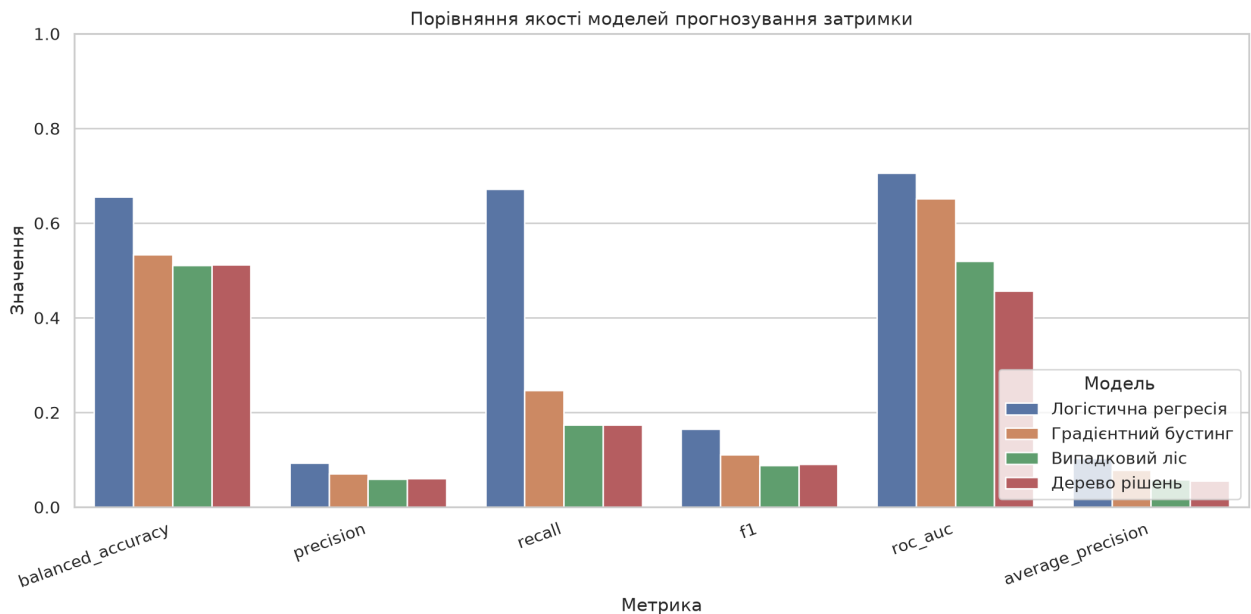


Рисунок 4.1 - Порівняння якості моделей

Найкращий результат за `average_precision`, `ROC-AUC`, `balanced_accuracy` та `F1` показала логістична регресія. Це означає, що в доступних ознаках присутній переважно лінійний сигнал, а складніші ансамблеві моделі на часовому тестовому періоді не дали переваги.

Водночас абсолютні значення `precision` залишаються низькими. Це пояснюється двома причинами. По-перше, клас затримок становить лише 8.11 % усіх замовлень. По-друге, у датасеті немає частини важливих операційних факторів: перевізника, складу, фактичної завантаженості маршруту, святкових періодів, погодних умов та внутрішніх логістичних подій.

4.5 Аналіз найкращої моделі

Найкращою моделлю стала логістична регресія з порогом 0.749, підібраним на валідаційній вибірці. На тестовій вибірці вона має:

- $\text{ROC-AUC} = 0.705$;
- $\text{average_precision} = 0.104$;
- $\text{balanced_accuracy} = 0.655$;
- $\text{recall} = 0.672$;
- $\text{precision} = 0.094$;
- $\text{F1} = 0.165$.

Матрицю помилок найкращої моделі наведено на рисунку 4.2.

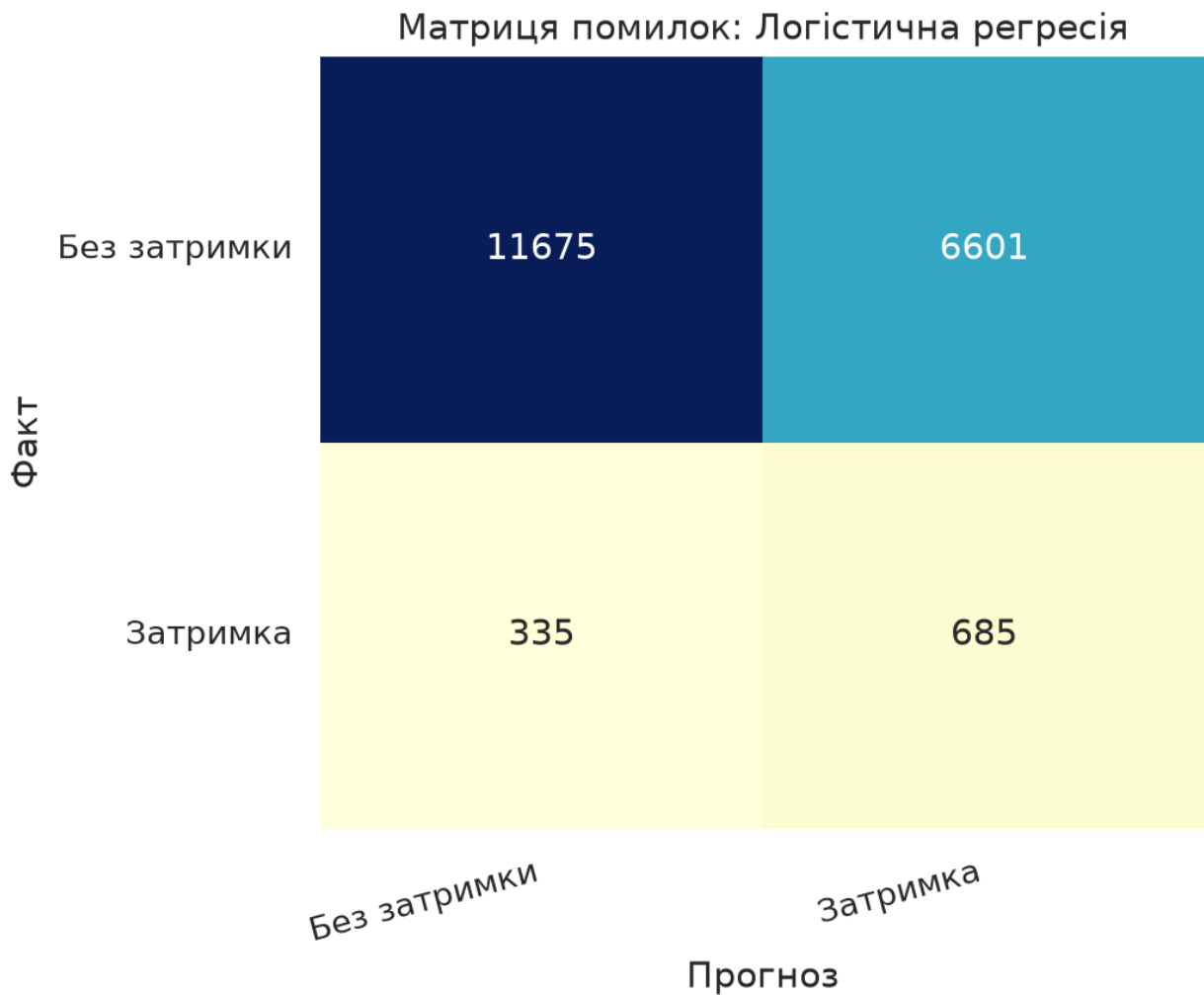


Рисунок 4.2 - Матриця помилок логістичної регресії

Модель знаходить приблизно 67 % фактично запізнених доставок, але при цьому формує багато хибних попереджень. Така поведінка може бути прийнятною для сценарію раннього моніторингу, де важливо не пропустити ризикові замовлення, але не підходить для автоматичного застосування штрафів або жорстких операційних рішень.

Найсильніші коефіцієнти логістичної регресії наведено на рисунку 4.3.

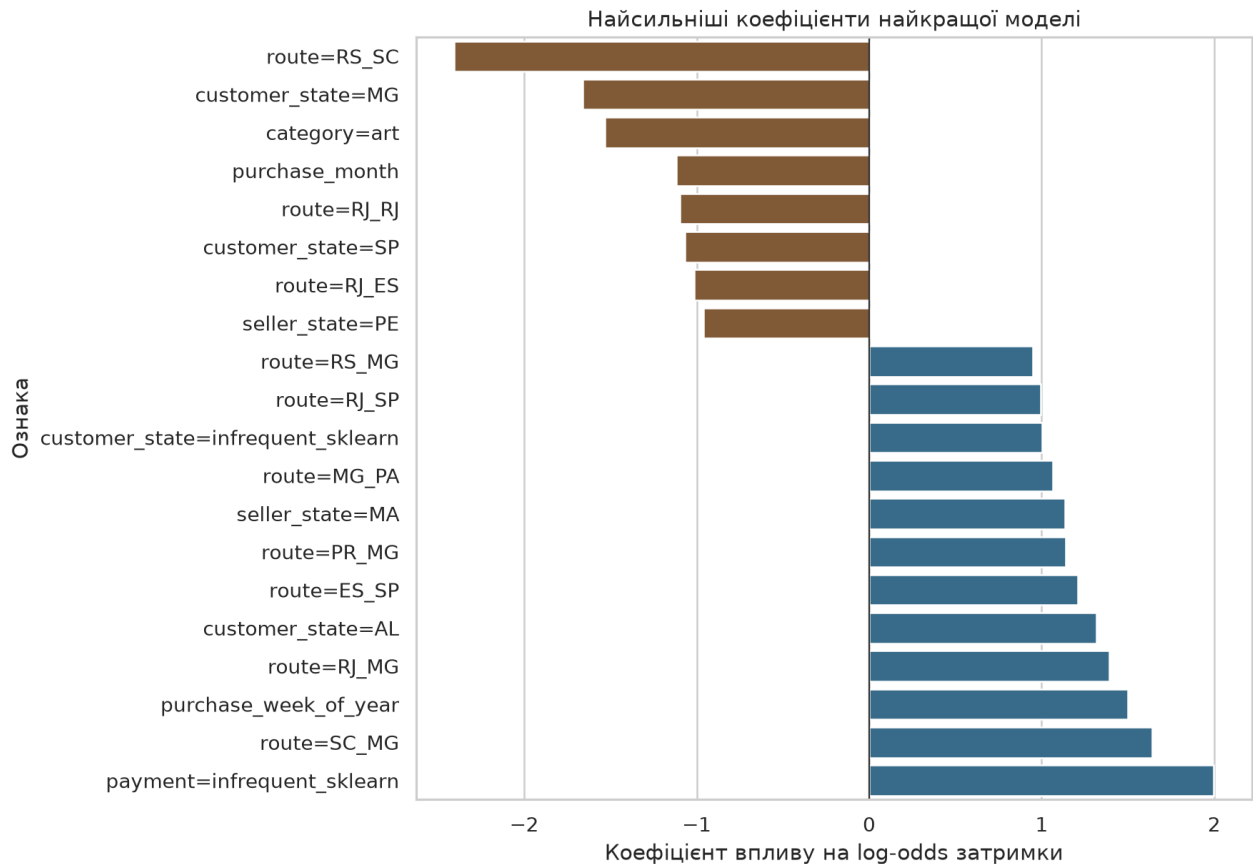


Рисунок 4.3 - Найсильніші коефіцієнти логістичної регресії. Додатні коефіцієнти збільшують log-odds затримки, від'ємні - зменшують. Серед впливових ознак присутні конкретні маршрути між штатами, штат покупця, тиждень року, типи оплати, а також окремі категорії товарів. Це узгоджується з предметною областю: ризик затримки залежить не лише від відстані, а й від регіональної логістичної структури та сезонно-операційних факторів.

4.6 Інтерпретація результатів

Порівняння моделей показало, що:

33. Географічні та логістичні ознаки містять корисний, але неповний сигнал про майбутню затримку.
34. Часовий поділ даних є важливим: моделі, які можуть добре працювати при випадковому поділі, не обов'язково стабільні на майбутньому періоді. Зокрема, дерево рішень на тестовому періоді має $ROC-AUC = 0.457$, тобто ранжує гірше за випадковий вибір, що є прямим наслідком розподільного зсуву в часі.

35. Висока ассигасу не означає високої якості для задачі з рідкісним класом. Дерево рішень і випадковий ліс мають ассигасу понад 0.81, але низький recall і низьку average precision.
36. Логістична регресія виявилася найкращою за здатністю ранжувати ризикові замовлення та знаходити більшу частину затримок.
37. Для продукційного використання модель треба доповнити ознаками перевізників, складів, фактичних маршрутів, святкових днів і поточного навантаження логістики.

Практичний спосіб використання моделі - не автоматичне рішення «затримка / не затримка», а ризиковий скоринг. Наприклад, замовлення з найбільшою прогновною ймовірністю затримки можуть потрапляти до окремої черги моніторингу або отримувати уточнений прогноз доставки.

ВИСНОВКИ

У курсовій роботі виконано аналіз та прогнозування ймовірності затримки доставки замовлення на основі логістичних параметрів і географічного розташування. Як джерело даних використано публічний датасет Olist і результати попередньої лабораторної роботи зі створення SQLite-сховища. Було сформовано аналітичну вибірку на рівні замовлення з 96 476 записів. Частка замовлень із затримкою становить 8.11 %, що робить задачу незбалансованою. Для уникнення витoku даних із моделювання вилучено фактичну тривалість доставки та фактичну кількість днів затримки. Побудовано та порівняно чотири моделі: логістичну регресію, дерево рішень, випадковий ліс і градієнтний бустинг. Найкращою стала логістична регресія з $\text{ROC-AUC} = 0.705$, $\text{average_precision} = 0.104$, $\text{balanced_accuracy} = 0.655$ і $\text{recall} = 0.672$. Це свідчить, що модель здатна виділяти групу підвищеного ризику, але точність позитивних спрацювань залишається низькою. Основний практичний висновок: доступні в Olist ознаки достатні для попереднього ризикового скорингу, але недостатні для високоточного прогнозування кожної конкретної затримки. Для покращення якості потрібні додаткові операційні дані: перевізник, склад, фактичний маршрут, святкові дні, погодні умови, завантаженість логістичних центрів та історія виконання продавців. Поставлені задачі виконано: предметну область описано, джерела даних проаналізовано, наявне сховище використано як основу, аналітичну вибірку сформовано, моделі побудовано та порівняно, результати інтерпретовано. Мету курсової роботи досягнуто.

ПЕРЕЛІК ПОСИЛАНЬ

1. Olist. Brazilian E-Commerce Public Dataset by Olist. Kaggle. URL: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce> (дата звернення: 16.05.2026).
2. Hosmer D. W., Lemeshow S., Sturdivant R. X. Applied Logistic Regression. 3rd ed. 2013.
3. Breiman L. Random Forests. Machine Learning. 2001. URL: <https://doi.org/10.1023/A:1010933404324>.
4. Friedman J. H. Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics. URL: <https://doi.org/10.1214/aos/1013203451>.
5. Saito T., Rehmsmeier M. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. URL: <https://doi.org/10.1371/journal.pone.0118432>.
6. scikit-learn developers. User Guide: Model evaluation. URL: https://scikit-learn.org/stable/modules/model_evaluation.html (дата звернення: 16.05.2026).
7. scikit-learn developers. LogisticRegression documentation. URL: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html (дата звернення: 16.05.2026).

ДОДАТОК А. ТЕКСТИ ПРОГРАМНОГО КОДУ

```

import json
import math
import sqlite3
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    precision_recall_curve,
    recall_score,
    roc_auc_score,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier

ROOT = Path(__file__).resolve().parents[2]
DB_PATH = ROOT / "lab_olist_warehouse" / "output" / "olist_dw.sqlite"
OUT_DIR = ROOT / "course_work" / "assets"
REPORT_TABLES_PATH = OUT_DIR / "model_metrics.csv"
DATASET_PATH = OUT_DIR / "delivery_delay_dataset.csv"
SUMMARY_PATH = OUT_DIR / "analysis_summary.json"

RANDOM_STATE = 42

def haversine_km(lat1: pd.Series, lon1: pd.Series, lat2: pd.Series, lon2: pd.Series) ->
pd.Series:
    radius_km = 6371.0
    lat1_rad = np.radians(lat1.astype(float))
    lon1_rad = np.radians(lon1.astype(float))
    lat2_rad = np.radians(lat2.astype(float))

```

```

lon2_rad = np.radians(lon2.astype(float))
dlat = lat2_rad - lat1_rad
dlon = lon2_rad - lon1_rad
a = np.sin(dlat / 2) ** 2 + np.cos(lat1_rad) * np.cos(lat2_rad) * np.sin(dlon / 2) **
2
c = 2 * np.arcsin(np.sqrt(a))
return radius_km * c

def load_dataset() -> pd.DataFrame:
    query = """
    WITH payment_agg AS (
        SELECT
            p.order_id,
            SUM(p.payment_value) AS payment_value,
            AVG(p.payment_installments) AS payment_installments,
            COUNT(*) AS payment_count,
            GROUP_CONCAT(DISTINCT pt.payment_type) AS payment_types
        FROM fact_payments p
        LEFT JOIN dim_payment_type pt ON pt.payment_type_key = p.payment_type_key
        GROUP BY p.order_id
    ),
    item_agg AS (
        SELECT
            f.order_id,
            COUNT(*) AS order_items_count,
            SUM(f.price) AS order_price_sum,
            SUM(f.freight_value) AS freight_sum,
            SUM(f.total_item_value) AS total_item_value_sum,
            AVG(f.price) AS item_price_avg,
            AVG(f.freight_value) AS freight_avg,
            AVG(prod.product_weight_g) AS product_weight_g_avg,
            AVG(prod.product_length_cm) AS product_length_cm_avg,
            AVG(prod.product_height_cm) AS product_height_cm_avg,
            AVG(prod.product_width_cm) AS product_width_cm_avg,
            AVG(prod.product_name_length) AS product_name_length_avg,
            AVG(prod.product_description_length) AS product_description_length_avg,
            AVG(prod.product_photos_qty) AS product_photos_qty_avg,
            MIN(COALESCE(cat.category_name_english, 'unknown')) AS primary_category,
            MIN(cust_loc.state) AS customer_state,
            MIN(cust_loc.city) AS customer_city,
            MIN(cust_loc.avg_lat) AS customer_lat,
            MIN(cust_loc.avg_lng) AS customer_lng,
            MIN(seller_loc.state) AS seller_state,
            MIN(seller_loc.city) AS seller_city,
            MIN(seller_loc.avg_lat) AS seller_lat,
            MIN(seller_loc.avg_lng) AS seller_lng,
            MIN(date_dim.year) AS purchase_year,

```

```

        MIN(date_dim.month) AS purchase_month,
        MIN(date_dim.day_of_week) AS purchase_day_of_week,
        MIN(date_dim.week_of_year) AS purchase_week_of_year,
        MIN(date_dim.full_date) AS purchase_date,
        MIN(ord.order_purchase_timestamp) AS purchase_timestamp,
        MIN(ord.order_approved_at) AS approved_timestamp,
        MIN(ord.order_estimated_delivery_date) AS estimated_delivery_timestamp,
        MIN(raw_item.shipping_limit_date) AS shipping_limit_timestamp,
        MAX(f.was_delivered_late) AS was_delivered_late,
        MAX(f.delivery_delay_days) AS delivery_delay_days,
        MAX(f.days_to_delivery) AS days_to_delivery
    FROM fact_order_items f
    LEFT JOIN stg_orders ord ON ord.order_id = f.order_id
    LEFT JOIN stg_order_items raw_item
        ON raw_item.order_id = f.order_id
        AND raw_item.order_item_id = f.order_item_id
    LEFT JOIN dim_product prod ON prod.product_key = f.product_key
    LEFT JOIN dim_category cat ON cat.category_key = prod.category_key
    LEFT JOIN dim_location cust_loc ON cust_loc.location_key =
f.customer_location_key
        LEFT JOIN dim_location seller_loc ON seller_loc.location_key =
f.seller_location_key
        LEFT JOIN dim_date date_dim ON date_dim.date_key = f.purchase_date_key
    WHERE f.was_delivered_late IS NOT NULL
    GROUP BY f.order_id
)
SELECT
    item_agg.*,
    COALESCE(payment_agg.payment_value, item_agg.total_item_value_sum) AS
payment_value,
    payment_agg.payment_installments,
    COALESCE(payment_agg.payment_count, 0) AS payment_count,
    COALESCE(payment_agg.payment_types, 'unknown') AS payment_types
FROM item_agg
LEFT JOIN payment_agg ON payment_agg.order_id = item_agg.order_id;
"""

with sqlite3.connect(f"file:{DB_PATH}?mode=ro", uri=True, timeout=15) as conn:
    df = pd.read_sql_query(query, conn)

    df["purchase_date"] = pd.to_datetime(df["purchase_date"], errors="coerce")
    df["purchase_timestamp"] = pd.to_datetime(df["purchase_timestamp"], errors="coerce")
    df["approved_timestamp"] = pd.to_datetime(df["approved_timestamp"], errors="coerce")
    df["estimated_delivery_timestamp"] =
pd.to_datetime(df["estimated_delivery_timestamp"], errors="coerce")
    df["shipping_limit_timestamp"] = pd.to_datetime(df["shipping_limit_timestamp"],
errors="coerce")
    df["estimated_delivery_days"] = (

```

```

        df["estimated_delivery_timestamp"] - df["purchase_timestamp"]
    ).dt.total_seconds() / 86400
    df["shipping_limit_days"] = (
        df["shipping_limit_timestamp"] - df["purchase_timestamp"]
    ).dt.total_seconds() / 86400
    df["approval_delay_hours"] = (
        df["approved_timestamp"] - df["purchase_timestamp"]
    ).dt.total_seconds() / 3600
    df["estimated_delivery_day_of_week"] =
df["estimated_delivery_timestamp"].dt.dayofweek
    df["shipping_limit_day_of_week"] = df["shipping_limit_timestamp"].dt.dayofweek
    df["same_state"] = (df["customer_state"] == df["seller_state"]).astype(int)
    df["freight_to_price_ratio"] = df["freight_sum"] / df["order_price_sum"].replace(0,
np.nan)
    df["product_volume_cm3_avg"] = (
        df["product_length_cm_avg"] * df["product_height_cm_avg"] *
df["product_width_cm_avg"]
    )
    df["distance_km"] = haversine_km(
        df["customer_lat"],
        df["customer_lng"],
        df["seller_lat"],
        df["seller_lng"],
    )
    df["distance_km"] = df["distance_km"].replace([np.inf, -np.inf], np.nan)
    df["route_state_pair"] = df["seller_state"].fillna("NA") + "_" +
df["customer_state"].fillna("NA")
    df["payment_types"] = df["payment_types"].fillna("unknown")
    df["primary_category"] = df["primary_category"].fillna("unknown")
    return df

```

```

def make_features(df: pd.DataFrame) -> tuple[pd.DataFrame, pd.Series, list[str],
list[str]]:
    leakage_columns = {
        "order_id",
        "purchase_date",
        "purchase_timestamp",
        "approved_timestamp",
        "estimated_delivery_timestamp",
        "shipping_limit_timestamp",
        "was_delivered_late",
        "delivery_delay_days",
        "days_to_delivery",
        "customer_lat",
        "customer_lng",
        "seller_lat",
        "seller_lng",
    }

```

```

        "customer_city",
        "seller_city",
    }
    categorical_features = [
        "primary_category",
        "customer_state",
        "seller_state",
        "route_state_pair",
        "payment_types",
    ]
    numeric_features = [
        col
        for col in df.columns
        if col not in leakage_columns and col not in categorical_features
    ]
    X = df[numeric_features + categorical_features].copy()
    y = df["was_delivered_late"].astype(int)
    return X, y, numeric_features, categorical_features

def chronological_split(
    df: pd.DataFrame,
    validation_ratio: float = 0.1,
    test_ratio: float = 0.2,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    ordered = df.sort_values(["purchase_date", "order_id"]).index.to_numpy()
    test_start = int(len(ordered) * (1 - test_ratio))
    validation_start = int(len(ordered) * (1 - test_ratio - validation_ratio))
    return ordered[:validation_start], ordered[validation_start:test_start],
    ordered[test_start:]

def build_preprocessor(numeric_features: list[str], categorical_features: list[str]) ->
ColumnTransformer:
    numeric_pipe = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]
    )
    categorical_pipe = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", min_frequency=50,
sparse_output=False)),
        ]
    )
    return ColumnTransformer(

```

```

        transformers=[
            ("num", numeric_pipe, numeric_features),
            ("cat", categorical_pipe, categorical_features),
        ]
    )

def model_scores(model: Pipeline, X: pd.DataFrame) -> np.ndarray:
    if hasattr(model, "predict_proba"):
        return model.predict_proba(X)[:, 1]
    return model.decision_function(X)

def select_threshold(y_true: pd.Series, y_score: np.ndarray) -> float:
    precision, recall, thresholds = precision_recall_curve(y_true, y_score)
    if len(thresholds) == 0:
        return 0.5
    f1_values = 2 * precision[:-1] * recall[:-1] / np.maximum(precision[:-1] + recall[:-1], 1e-12)
    best_idx = int(np.nanargmax(f1_values))
    return float(thresholds[best_idx])

def predict_with_threshold(model: Pipeline, X: pd.DataFrame, threshold: float) -> np.ndarray:
    return (model_scores(model, X) >= threshold).astype(int)

def evaluate_model(
    name: str,
    model: Pipeline,
    threshold: float,
    X_test: pd.DataFrame,
    y_test: pd.Series,
) -> dict[str, float | str]:
    y_score = model_scores(model, X_test)
    y_pred = (y_score >= threshold).astype(int)

    metrics = {
        "model": name,
        "threshold": threshold,
        "accuracy": accuracy_score(y_test, y_pred),
        "balanced_accuracy": balanced_accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, zero_division=0),
        "recall": recall_score(y_test, y_pred, zero_division=0),
        "f1": f1_score(y_test, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_test, y_score),
        "average_precision": average_precision_score(y_test, y_score),
    }

```

```

        "predicted_positive_rate": float(np.mean(y_pred)),
    }
    return metrics

def plot_target_distribution(df: pd.DataFrame) -> None:
    plt.figure(figsize=(8, 5))
    ax = sns.countplot(data=df, x="was_delivered_late", hue="was_delivered_late",
palette=["#41788f", "#d95f02"], legend=False)
    ax.set_title("Розподіл цільової ознаки затримки доставки")
    ax.set_xlabel("Замовлення доставлено із затримкою")
    ax.set_ylabel("Кількість замовлень")
    ax.set_xticks([0, 1], ["Ні", "Так"])
    for container in ax.containers:
        ax.bar_label(container)
    plt.tight_layout()
    plt.savefig(OUT_DIR / "target_distribution.png", dpi=180)
    plt.close()

def plot_late_by_state(df: pd.DataFrame) -> None:
    state_stats = (
        df.groupby("customer_state")
        .agg(orders=("order_id", "count"), late_rate=("was_delivered_late", "mean"))
        .query("orders >= 100")
        .sort_values("late_rate", ascending=False)
        .head(15)
        .reset_index()
    )
    plt.figure(figsize=(10, 6))
    ax = sns.barplot(data=state_stats, y="customer_state", x="late_rate",
color="#8c5a2b")
    ax.set_title("Найвища частка затримок за штатами покупців")
    ax.set_xlabel("Частка замовлень із затримкою")
    ax.set_ylabel("Штат покупця")
    ax.xaxis.set_major_formatter(lambda x, _: f"{x:.0%}")
    plt.tight_layout()
    plt.savefig(OUT_DIR / "late_rate_by_customer_state.png", dpi=180)
    plt.close()

def plot_distance_effect(df: pd.DataFrame) -> None:
    prepared = df.dropna(subset=["distance_km"]).copy()
    prepared["distance_bin"] = pd.qcut(prepared["distance_km"], q=10, duplicates="drop")
    distance_stats = (
        prepared.groupby("distance_bin", observed=False)
        .agg(distance_km=("distance_km", "median"), late_rate=("was_delivered_late",
"mean"), orders=("order_id", "count"))

```

```

        .reset_index(drop=True)
    )
    plt.figure(figsize=(9, 5))
    ax = sns.lineplot(data=distance_stats, x="distance_km", y="late_rate", marker="o",
color="#2a6f97")
    ax.set_title("Залежність частки затримок від відстані між продавцем і покупцем")
    ax.set_xlabel("Медіанна відстань у групі, км")
    ax.set_ylabel("Частка затримок")
    ax.yaxis.set_major_formatter(lambda x, _: f"{x:.0%}")
    plt.tight_layout()
    plt.savefig(OUT_DIR / "late_rate_by_distance.png", dpi=180)
    plt.close()

def plot_metrics(metrics_df: pd.DataFrame) -> None:
    long_df = metrics_df.melt(
        id_vars="model",
        value_vars=["balanced_accuracy", "precision", "recall", "f1", "roc_auc",
"average_precision"],
        var_name="metric",
        value_name="value",
    )
    plt.figure(figsize=(12, 6))
    ax = sns.barplot(data=long_df, x="metric", y="value", hue="model")
    ax.set_title("Порівняння якості моделей прогнозування затримки")
    ax.set_xlabel("Метрика")
    ax.set_ylabel("Значення")
    ax.set_ylim(0, 1)
    ax.legend(title="Модель", loc="lower right")
    plt.xticks(rotation=20, ha="right")
    plt.tight_layout()
    plt.savefig(OUT_DIR / "model_metrics_comparison.png", dpi=180)
    plt.close()

def plot_confusion(best_name: str, best_model: Pipeline, threshold: float, X_test:
pd.DataFrame, y_test: pd.Series) -> None:
    y_pred = predict_with_threshold(best_model, X_test, threshold)
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(6, 5))
    ax = sns.heatmap(cm, annot=True, fmt="d", cmap="YlGnBu", cbar=False)
    ax.set_title(f"Матриця помилок: {best_name}")
    ax.set_xlabel("Прогноз")
    ax.set_ylabel("Факт")
    ax.set_xticklabels(["Без затримки", "Затримка"], rotation=15, ha="right")
    ax.set_yticklabels(["Без затримки", "Затримка"], rotation=0)
    plt.tight_layout()
    plt.savefig(OUT_DIR / "best_model_confusion_matrix.png", dpi=180)

```

```

plt.close()

def plot_feature_importance(best_model: Pipeline, numeric_features: list[str],
categorical_features: list[str]) -> None:
    classifier = best_model.named_steps["classifier"]

    preprocessor = best_model.named_steps["preprocessor"]
    feature_names = preprocessor.get_feature_names_out()

    if hasattr(classifier, "feature_importances_"):
        importance_values = classifier.feature_importances_
        title = "Найважливіші ознаки найкращої моделі"
        x_label = "Важливість ознаки"
        output_name = "feature_importance.png"
        sort_col = "importance_abs"
    elif hasattr(classifier, "coef_"):
        importance_values = classifier.coef_[0]
        title = "Найсильніші коефіцієнти найкращої моделі"
        x_label = "Коефіцієнт впливу на log-odds затримки"
        output_name = "feature_importance.png"
        sort_col = "importance_abs"
    else:
        return

    importance = pd.DataFrame({"feature": feature_names, "importance":
importance_values})
    importance["importance_abs"] = importance["importance"].abs()
    importance["feature"] = (
        importance["feature"]
        .str.replace("num_", "", regex=False)
        .str.replace("cat_", "", regex=False)
        .str.replace("primary_category_", "category=", regex=False)
        .str.replace("customer_state_", "customer_state=", regex=False)
        .str.replace("seller_state_", "seller_state=", regex=False)
        .str.replace("route_state_pair_", "route=", regex=False)
        .str.replace("payment_types_", "payment=", regex=False)
    )
    top = importance.sort_values(sort_col,
ascending=False).head(20).sort_values("importance")
    plt.figure(figsize=(10, 7))
    palette = ["#2a6f97" if value > 0 else "#8c5a2b" for value in top["importance"]]
    ax = sns.barplot(data=top, y="feature", x="importance", hue="feature",
palette=palette, legend=False)
    ax.axvline(0, color="#333333", linewidth=0.9)
    ax.set_title(title)
    ax.set_xlabel(x_label)
    ax.set_ylabel("Ознака")

```

```

plt.tight_layout()
plt.savefig(OUT_DIR / output_name, dpi=180)
plt.close()

def main() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    sns.set_theme(style="whitegrid", font="DejaVu Sans")

    df = load_dataset()
    df.to_csv(DATASET_PATH, index=False)

    X, y, numeric_features, categorical_features = make_features(df)
    train_idx, validation_idx, test_idx = chronological_split(df)
    X_train, X_test = X.loc[train_idx], X.loc[test_idx]
    y_train, y_test = y.loc[train_idx], y.loc[test_idx]
    X_validation, y_validation = X.loc[validation_idx], y.loc[validation_idx]

    class_balance = y_train.value_counts(normalize=True).to_dict()
    neg_share = class_balance.get(0, 0.0)
    pos_share = class_balance.get(1, 0.0)
    pos_weight = neg_share / pos_share if pos_share else 1.0

    models = {
        "Логістична перспєція": LogisticRegression(
            max_iter=3000,
            class_weight="balanced",
            solver="lbfgs",
            random_state=RANDOM_STATE,
        ),
        "Дерево рішень": DecisionTreeClassifier(
            max_depth=12,
            min_samples_leaf=50,
            class_weight="balanced",
            random_state=RANDOM_STATE,
        ),
        "Випадковий ліс": RandomForestClassifier(
            n_estimators=250,
            max_depth=16,
            min_samples_leaf=20,
            class_weight="balanced_subsample",
            n_jobs=-1,
            random_state=RANDOM_STATE,
        ),
        "Градiєнтний бустинг": HistGradientBoostingClassifier(
            max_iter=180,
            learning_rate=0.08,
            max_leaf_nodes=31,

```

```

        l2_regularization=0.05,
        class_weight={0: 1.0, 1: float(pos_weight)},
        random_state=RANDOM_STATE,
    ),
}

fitted_models: dict[str, Pipeline] = {}
thresholds: dict[str, float] = {}
metrics = []
reports = {}
for name, classifier in models.items():
    pipeline = Pipeline(
        steps=[
            ("preprocessor", build_preprocessor(numeric_features,
categorical_features)),
            ("classifier", classifier),
        ]
    )
    pipeline.fit(X_train, y_train)
    fitted_models[name] = pipeline
    thresholds[name] = select_threshold(y_validation, model_scores(pipeline,
X_validation))
    metrics.append(evaluate_model(name, pipeline, thresholds[name], X_test, y_test))
    reports[name] = classification_report(
        y_test,
        predict_with_threshold(pipeline, X_test, thresholds[name]),
        output_dict=True,
        zero_division=0,
    )

metrics_df = pd.DataFrame(metrics).sort_values("average_precision", ascending=False)
metrics_df.to_csv(REPORT_TABLES_PATH, index=False)

best_name = metrics_df.iloc[0]["model"]
best_model = fitted_models[str(best_name)]
best_threshold = thresholds[str(best_name)]

plot_target_distribution(df)
plot_late_by_state(df)
plot_distance_effect(df)
plot_metrics(metrics_df)
plot_confusion(str(best_name), best_model, best_threshold, X_test, y_test)
plot_feature_importance(best_model, numeric_features, categorical_features)

summary = {
    "source_database": str(DB_PATH.relative_to(ROOT)),
    "rows": int(len(df)),

```

```

"columns": int(len(df.columns)),
"target_positive_rate": float(df["was_delivered_late"].mean()),
"train_rows": int(len(train_idx)),
"validation_rows": int(len(validation_idx)),
"test_rows": int(len(test_idx)),
"train_period": [
    str(df.loc[train_idx, "purchase_date"].min().date()),
    str(df.loc[train_idx, "purchase_date"].max().date()),
],
"validation_period": [
    str(df.loc[validation_idx, "purchase_date"].min().date()),
    str(df.loc[validation_idx, "purchase_date"].max().date()),
],
"test_period": [
    str(df.loc[test_idx, "purchase_date"].min().date()),
    str(df.loc[test_idx, "purchase_date"].max().date()),
],
"late_delay_days_mean": float(df.loc[df["was_delivered_late"] == 1,
"delivery_delay_days"].mean()),
"late_delay_days_median": float(df.loc[df["was_delivered_late"] == 1,
"delivery_delay_days"].median()),
"distance_km_median": float(df["distance_km"].median()),
"distance_km_late_median": float(df.loc[df["was_delivered_late"] == 1,
"distance_km"].median()),
"distance_km_on_time_median": float(df.loc[df["was_delivered_late"] == 0,
"distance_km"].median()),
"best_model": str(best_name),
"best_threshold": float(best_threshold),
"metrics": metrics_df.to_dict(orient="records"),
"numeric_features": numeric_features,
"categorical_features": categorical_features,
"classification_reports": reports,
}
SUMMARY_PATH.write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding="utf-8")

print(json.dumps(summary, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```